
Riprogettazione di un sistema text-to-SQL in architettura agentica

Daniele Lurani

60/73/65357

Università degli Studi di Cagliari

21 Settembre, 2026

ABSTRACT

I sistemi text-to-SQL traducono domande in linguaggio naturale in query eseguibili, ma una query sintatticamente valida può rispondere alla domanda sbagliata senza che il database segnali alcun errore. Questa relazione descrive la trasformazione di una pipeline text-to-SQL in un sistema agentico costruito con LangGraph, nel quale il modello linguistico sceglie da solo quali strumenti usare per leggere lo schema, campionare i dati ed eseguire le query, e sottopone la propria risposta a una verifica prima di restituirla. Un primo tentativo di affidare quella verifica a un secondo modello linguistico è fallito proprio sul caso principale, perché il verificatore non disponeva delle informazioni necessarie per riconoscere l'errore. Il controllo è stato quindi riscritto come regola deterministica dentro gli strumenti, lasciando al modello i soli casi che richiedono interpretazione. Su un database di valutazione con quattro tabelle e circa millecinquecento righe, il sistema agentico ha risposto correttamente in quindici esecuzioni su quindici, contro tre su quindici della pipeline di partenza a parità di modello linguistico.

1 Introduzione

Interrogare un database richiede di conoscere SQL e di sapere com'è fatto lo schema. I sistemi text-to-SQL servono a togliere questo requisito, traducendo una domanda scritta in linguaggio naturale in una query da eseguire. Con i modelli linguistici recenti la traduzione riesce quasi sempre a produrre SQL sintatticamente valido.

Il problema è che SQL valido non vuol dire SQL corretto. Una query può essere eseguita senza errori e rispondere a una domanda diversa da quella posta, oppure filtrare su un valore che nei dati non esiste. In entrambi i casi il database non segnala nulla, e chi ha fatto la domanda riceve un numero sbagliato senza avere motivo di dubitarne. È da qui che parte il lavoro descritto in questa relazione.

Il progetto ha l'obiettivo di trasformare un sistema text-to-SQL già funzionante in un sistema agentico. Il sistema di partenza era una pipeline fissa che generava una query, la eseguiva e, in caso di errore, rimandava il messaggio del database al modello perché si correggesse. Il meccanismo funziona, ma il modello non decide nulla e non ha modo di guardare il database prima di scrivere la query.

La relazione segue l'ordine in cui il lavoro è stato svolto. I capitoli 2 e 3 descrivono il sistema di partenza e spiegano perché non può essere considerato agentic. Il capitolo 4 confronta i framework disponibili e motiva la scelta. I capitoli 5 e 6 descrivono la progettazione e la realizzazione del sistema nuovo. Il capitolo 7 racconta un tentativo che non ha funzionato, cioè affidare a un modello linguistico la verifica della correttezza delle risposte, e la soluzione che ne è nata. Il capitolo 8 riporta i risultati, il 9 i limiti e il 10 le conclusioni.

2 Il sistema di partenza

2.1 Funzionamento

Il progetto di partenza traduce una domanda in linguaggio naturale in una query SQL, la esegue su un database SQLite e restituisce le righe ottenute. È composto da cinque file.

`schema_utils.py` legge la struttura del database e la trasforma in un blocco di testo che elenca tabelle, colonne e chiavi esterne. Questo testo viene inserito nel prompt come contesto, perché il modello non ha altro modo di sapere com'è fatto il database.

`nl2sql_agent.py` invia la richiesta al modello linguistico e ne estrae la query, che il modello restituisce dentro un blocco di codice. La classe mantiene lo storico dei messaggi scambiati.

`db_executor.py` esegue la query, ma non nel processo principale. La esegue in un sottoprocesso separato, così una query che si blocca può essere interrotta con un timeout senza fermare il resto del programma. `_db_worker.py` è lo script eseguito da quel sottoprocesso.

`main.py` contiene il ciclo che lega tutto. Genera la query, la esegue e, se l'esecuzione fallisce, ripete fino a un massimo di quattro tentativi.

```
domanda --> [agente: genera SQL] --> [executor: esegue] --> righe
                ^                               |
                |___ errore del DB ___| (max 4 tentativi)
```

Listing 1: Il flusso del sistema di partenza.

La parte più interessante del sistema è il modo in cui gestisce gli errori. Quando una query fallisce, il messaggio di errore del database non viene scartato ma aggiunto alla stessa conversazione. Al tentativo successivo il modello vede la query che aveva scritto e l'errore esatto che ha causato, quindi non riparte da zero ma corregge. Questo meccanismo è stato mantenuto anche nel sistema nuovo.

2.2 Difetti rilevati

Durante l'analisi del codice sono emersi alcuni problemi, alcuni dei quali rilevanti per il seguito del lavoro.

Il primo riguarda il controllo sulle scritture. `db_executor.py` rifiuta le query che iniziano con una parola presente in una lista, che contiene `DROP`, `DELETE`, `UPDATE`, `ALTER`, `INSERT` e `TRUNCATE`. Il controllo guarda però solo la prima parola, e la lista non comprende tutti i comandi che modificano il database.

Query provata	Esito
DROP TABLE customers	bloccata
-- commento	eseguita
DROP TABLE customers	
CREATE TABLE evil (x INT)	eseguita
REPLACE INTO customers VALUES (...)	eseguita
ATTACH DATABASE '/etc/passwd' AS p	eseguita
PRAGMA table_info(customers)	eseguita

Table 1: Query provate contro il controllo del sistema di partenza. Solo la prima viene effettivamente bloccata.

Un secondo problema riguarda il parametro `allow_writes`. Quando è attivo la query di scrittura viene eseguita, ma la connessione viene chiusa senza chiamare `commit`, quindi la modifica va persa. La funzione non ha quindi mai funzionato.

Il terzo problema è l'assenza di un limite sulle righe restituite. Sul database di esempio non si nota, perché contiene sette righe in tutto, ma su una tabella grande il risultato verrebbe caricato per intero in memoria e inserito nel prompt.

Restano infine due imprecisioni minori. La documentazione di `main.py` indica una variabile d'ambiente diversa da quella effettivamente usata dal codice, e `nl2sql_agent.py` stampa l'intera conversazione a ogni chiamata al modello.

3 Perché non era un sistema agentic

3.1 Pipeline e agente

Prima di trasformare il sistema è stato necessario stabilire cosa distingue un agente da una normale sequenza di chiamate. Un sistema viene di solito considerato agentic quando il modello decide da solo cosa fare, può usare strumenti per procurarsi le informazioni che gli mancano, sa scomporre un compito in più passi, valuta il risultato del proprio lavoro e conserva memoria di quello che è già successo.

Il sistema di partenza non soddisfa nessuno di questi punti. Le decisioni non le prende il modello, perché la sequenza delle operazioni è scritta in un ciclo `for` dentro `main.py` e il modello viene chiamato sempre nello stesso momento per fare sempre la stessa cosa. Non ha inoltre alcuno strumento a disposizione. Riceve lo schema del database già pronto nel prompt e non può chiedere altro, quindi non può guardare quali valori contiene davvero una colonna né eseguire la query per conto suo.

Manca anche la pianificazione, perché ogni domanda produce una query e basta. Una richiesta come confrontare due gruppi di clienti e commentare la differenza non è esprimibile, dato che andrebbe divisa in più passi. Manca infine la memoria, visto che in modalità interattiva ogni domanda crea un oggetto `NL2SQLAgent` nuovo e una domanda che si riferisce alla precedente non funziona.

Resta la valutazione del risultato, che nel sistema di partenza riguarda solo gli errori di esecuzione. È il punto più importante dei cinque e viene ripreso nel paragrafo successivo.

Criterio	Sistema di partenza
Autonomia nelle decisioni	la sequenza è fissata in un ciclo <code>for</code>
Uso di strumenti	nessuno, il modello produce solo testo
Pianificazione	una domanda produce una sola query
Valutazione del risultato	solo gli errori di esecuzione
Memoria	ogni domanda crea un agente nuovo

Table 2: I criteri di agenticità applicati al sistema di partenza.

3.2 Successo non vuol dire correttezza

Il campo `result.success` vale vero quando SQLite ha eseguito la query senza sollevare eccezioni. Non dice però niente sul fatto che la query risponda alla domanda posta. Una query con la join sbagliata, con `COUNT` al posto di `SUM` o con un filtro dimenticato viene eseguita senza problemi e considerata un successo.

Il caso più chiaro riguarda i valori contenuti nelle colonne. Nel database di esempio la colonna `country` contiene codici come `IT` e `UK`, non i nomi estesi dei paesi. Alla domanda su quanti siano i clienti italiani il modello scrive `WHERE country = 'Italy'`, perché è la scrittura più naturale e nulla nello schema lo avverte del contrario. La query viene eseguita, restituisce zero righe e il sistema la considera riuscita. La risposta è zero ed è sbagliata.

Il meccanismo di autocorrezione descritto nel capitolo precedente non può intervenire, perché non c'è nessun errore da rimandare al modello. Questo è il problema da cui è partita la riprogettazione, e viene ripreso nel capitolo 7.

4 Scelta del framework

4.1 Criteri e candidati

I criteri per la scelta del framework sono stati fissati prima di guardare i candidati, per evitare di sceglierli in modo da giustificare una decisione già presa.

Il primo criterio è il costo, che doveva essere nullo trattandosi di un progetto universitario senza budget. Il secondo è il controllo esplicito sul flusso, cioè la possibilità di descrivere nel codice l'ordine delle operazioni invece di lasciarlo emergere dal comportamento del modello. Il terzo è la capacità di esprimere cicli, dato che sia l'autocorrezione sia la revisione della risposta sono cicli. Il quarto è il consumo di token, che su un piano gratuito è una risorsa limitata. Il quinto è la leggibilità dell'architettura, utile sia durante lo sviluppo sia per questa relazione.

I candidati considerati sono stati quattro. LangGraph descrive il sistema come un grafo di stati, con nodi e archi dichiarati nel codice. CrewAI organizza il lavoro fra agenti con ruoli distinti che si passano i risultati. PydanticAI è più leggero e fornisce output tipizzati, lasciando il ciclo agentic implicito nel framework. Il Claude Agent SDK mette a disposizione un agente già completo di strumenti.

Framework	Costo	Cicli e controllo del flusso	Architettura
LangGraph	gratuito	archi condizionali espliciti	diagrammabile
CrewAI	gratuito	solo con le Flows	implicita
PydanticAI	gratuito	da scrivere a mano	implicita
Claude Agent SDK	a pagamento	gestiti dal framework	implicita

Table 3: I framework confrontati sui criteri che si sono rivelati discriminanti.

4.2 La scelta e le esclusioni

La scelta è caduta su LangGraph, per una ragione che riguarda la forma del problema più che le caratteristiche del framework. Tradurre una domanda in SQL non è un lavoro da dividere fra competenze diverse, ma un ciclo stretto con una sola competenza, in cui si guarda lo schema, si scrive una query, la si esegue, si legge l'errore e si corregge. È quindi un problema di controllo di flusso, che è esattamente ciò che LangGraph descrive. Il grafo inoltre sa generare il proprio diagramma, usato come figura nel capitolo 5.

Il Claude Agent SDK è stato escluso perché richiede un'API a pagamento senza piano gratuito, quindi non rispetta il primo criterio.

CrewAI è stato escluso per il secondo e il quarto. Le Crew descrivono ruoli che si passano il testimone e non permettono di esprimere una condizione del tipo esegui, controlla il risultato e poi scegli il passo successivo, che è proprio il ciclo di autocorrezione. CrewAI mette a disposizione le Flows per colmare questa mancanza, ma usarle significa ricostruire una macchina a stati sopra un'astrazione pensata per altro, pagando in più il costo dei ruoli. Ogni chiamata a un agente CrewAI porta infatti con sé la descrizione del ruolo e dell'obiettivo, per circa mille token aggiuntivi, che su un piano gratuito da poche migliaia di token al minuto è un costo rilevante. CrewAI dichiara inoltre di non supportare Python 3.14, la versione installata sulla macchina di sviluppo, ma questo è un fatto contingente e non è stato considerato un argomento.

PydanticAI è stato scartato per il quinto criterio. Avrebbe portato a un risultato funzionante più in fretta e con meno codice, ma il ciclo agentico resta nascosto dentro il framework, quindi i due cicli del sistema andrebbero scritti a mano e l'architettura non sarebbe visibile né diagrammabile.

5 Progettazione del sistema agentico

5.1 L'architettura a grafo

Il sistema è descritto come un grafo con tre nodi e due cicli. La struttura è dichiarata nel codice, e il comando `main.py --diagram` la esporta in formato Mermaid a partire dal grafo compilato, quindi lo schema della figura corrisponde a ciò che viene effettivamente eseguito.



Il grafo è infine compilato con un checkpoint, che conserva lo stato di ogni conversazione. Questo permette di fare domande di seguito. Dopo aver chiesto quanti sono i clienti italiani, alla domanda *e per il Regno Unito?* l'agente non riesplora lo schema ma scrive direttamente la query, passando da quattro chiamate a una.

L'agente dispone di quattro strumenti, ognuno dei quali richiama funzioni che esistevano già nel sistema di partenza.

Strumento	Cosa fa
<code>list_tables</code>	elenca i nomi delle tabelle
<code>describe_table</code>	mostra colonne, tipi e chiavi di una tabella
<code>sample_rows</code>	mostra alcune righe reali di una tabella
<code>run_select</code>	esegue una query di lettura e restituisce le righe oppure l'errore del database

Table 4: Gli strumenti a disposizione dell'agente.

Ogni strumento ha una breve descrizione scritta in inglese. Quella descrizione non serve a chi legge il codice, ma viene inviata al modello insieme alla domanda a ogni chiamata. Il modello infatti non vede il codice Python degli strumenti, ma solo il loro nome, i parametri che accettano e appunto la descrizione, che è quindi l'unica cosa su cui può basarsi per capire quale strumento gli serve. Cambiarla cambia il comportamento dell'agente, quindi fa parte del programma a tutti gli effetti. Un esempio è la descrizione di `sample_rows`, che avverte il modello che una colonna può contenere IT invece di Italy. È questa frase a spingerlo a guardare i dati prima di filtrare.

6 Implementazione

6.1 Organizzazione del codice

Il progetto è diviso in due pacchetti. In `sql_agent` sta il sistema agentico, con la scelta del modello, gli strumenti, il grafo e le funzioni che leggono ed eseguono sul database. In `legacy` è conservato il sistema di partenza, che serve da termine di paragone per il confronto del capitolo 8. La separazione rende immediato distinguere cosa è stato scritto da cosa era già presente.

Buona parte del codice del sistema di partenza è stata riusata. Gli strumenti dell'agente non fanno altro che richiamare le funzioni di introspezione dello schema e di esecuzione che esistevano già. È stata mantenuta anche l'esecuzione in sottoprocesso, che era una scelta corretta, perché una query che si blocca viene interrotta con un timeout senza fermare il ciclo dell'agente.

Il modello linguistico non è fissato nel codice ma scelto da una variabile d'ambiente, fra due fornitori che offrono entrambi un piano gratuito. La ragione non è la flessibilità in sé, ma il fatto che i piani gratuiti limitano i token al minuto, e restare senza quota durante una dimostrazione bloccherebbe il sistema. Entrambi i fornitori sono stati provati sulle stesse domande.

6.2 Il controllo sulle scritture

Nel sistema di partenza le query venivano scritte da un modello chiamato una volta sola e passavano per un ciclo esterno. Ora è l'agente a decidere da solo quando eseguire una query, e può farlo più volte per ogni domanda, quindi il controllo su cosa viene eseguito diventa più importante. Il controllo è stato riscritto su tre strati, perché nessun controllo singolo è sufficiente.

Strato	Cosa ferma
Apertura in sola lettura	ogni scrittura, rifiutata direttamente da SQLite
Lista dei comandi permessi	tutto ciò che non inizia con SELECT o WITH , dopo aver tolto commenti e stringhe
Controllo sul resto della query	più istruzioni separate da punto e virgola, e scritture nascoste dopo un WITH

Table 5: I tre strati del controllo sulle query.

La differenza principale rispetto al sistema di partenza è che il secondo strato elenca ciò che è permesso invece di ciò che è vietato. Una lista di divieti è incompleta per costruzione, perché ogni comando che non viene in mente resta permesso, ed è esattamente quello che succedeva prima con **CREATE** e **ATTACH**. Una lista di permessi rifiuta invece per default tutto ciò che non riconosce.

I commenti e le stringhe vengono tolti prima di guardare la prima parola, altrimenti basterebbe un commento iniziale per nascondere un comando di scrittura. Allo stesso tempo il controllo non guarda mai il testo intero della query, così una parola come **DROP** scritta dentro una stringa o una colonna chiamata `updated_at` non vengono scambiate per scritture.

Un ultimo accorgimento riguarda il numero di righe restituite. Le query vengono troncate a un massimo e il risultato porta con sé l'indicazione di essere stato troncato, così l'agente sa di guardare un campione e non ne conta le righe come se fossero il totale.

7 Verifica semantica, un risultato negativo

7.1 Il problema e il primo tentativo

Nei capitoli precedenti è stato mostrato che una query può essere eseguita senza errori e restituire comunque una risposta sbagliata. Il sistema di partenza controllava solo che il database non sollevasse eccezioni, quindi una query valida ma mal formulata veniva considerata corretta. Serviva un controllo sulla risposta e non sull'esecuzione.

La prima soluzione è stata aggiungere al grafo un nodo che usa un secondo modello come revisore. Il nodo **verify** riceve la domanda, la query eseguita, le righe ottenute e la risposta proposta dall'agente, e decide se approvarla o rifiutarla. Nel prompt sono stati elencati i casi da rifiutare, per esempio una query che risponde a una domanda diversa, un risultato vuoto riportato senza indagare, o un totale calcolato su righe troncate.

Il verdetto doveva inizialmente essere un oggetto strutturato, con un campo per l'esito e uno per la motivazione. Questa strada è stata abbandonata perché il modello non la rispettava in modo affidabile, rispondendo a volte in prosa invece di produrre l'oggetto richiesto e altre volte restituendo la descrizione del formato al posto di un valore. Il verdetto è quindi diventato una sola riga di testo, che vale come approvazione solo se comincia con **APPROVED** e in ogni altro caso conta come rifiuto. Una risposta incomprensibile porta così a una revisione in più e non a un'approvazione per sbaglio.

Se il verificatore rifiuta, la motivazione torna all'agente, che riformula la risposta. Il numero di revisioni è limitato per evitare cicli infiniti.

7.2 Il fallimento del verificatore

Il verificatore è stato provato sul caso descritto nel capitolo 3. Gli è stata sottoposta una trascrizione in cui l'agente filtra su `country = 'Italy'`, ottiene zero righe e conclude che non ci sono clienti italiani. La risposta è sbagliata, perché nel database il paese è scritto `IT`.

Il verificatore ha approvato.

```
QUESTION: Quanti clienti italiani ci sono?  
called run_select({'sql': "SELECT COUNT(*) FROM customers WHERE country =  
'Italy'"})  
returned: COUNT(*)  
0  
(1 rows)  
PROPOSED ANSWER: Non ci sono clienti italiani nel database.
```

Listing 3: La trascrizione sottoposta al verificatore, che ha risposto **APPROVED**.

Il motivo è che il verificatore vede solo la query e le righe prodotte, e non ha mai osservato il contenuto della colonna. Non può quindi sapere che il valore `Italy` non compare nei dati. Con una query corretta e un risultato vuoto, la conclusione che non ci siano clienti italiani è plausibile.

Provato su altri casi, però, il verificatore si è comportato bene. Ha rifiutato una risposta che contava cinquanta righe di un campione troncato come se fossero il totale, e ne ha rifiutata una in cui la query contava i clienti italiani mentre la domanda chiedeva quanto avessero speso. Il problema non era quindi il verificatore, ma il tipo di controllo che gli veniva chiesto.

7.3 La soluzione deterministica

Il controllo mancante non richiede interpretazione. Se una query filtra su un valore testuale e non restituisce nulla, il risultato è ambiguo di per sé, e per rilevarlo basta una regola.

Il controllo è stato quindi spostato dentro lo strumento `run_select`. Quando una query non restituisce righe, lo strumento estrae i valori testuali usati nei filtri e li segnala all'agente.

```
No rows returned. This query filters on the literal value(s) 'Italy'.  
An empty result looks the same whether nothing matches or the filter  
value is simply not written that way in the data. Use sample_rows to  
check how the column is actually written before reporting this as a  
finding.
```

Listing 4: Il messaggio che lo strumento restituisce al posto di un risultato vuoto.

Rispetto al verificatore questa soluzione scatta sempre, non dipende da come il modello interpreta il prompt e non richiede una chiamata in più all'API. Arriva inoltre mentre l'agente sta ancora lavorando, invece che dopo che la risposta è stata formulata.

Il verificatore è stato mantenuto per i casi che richiedono interpretazione, cioè proprio quelli che aveva superato, come una query che risponde a una domanda diversa da quella posta o un totale calcolato su un campione. Le regole meccaniche stanno negli strumenti, il giudizio nel verificatore.

Da questo episodio è derivato un criterio applicato anche altrove nel progetto. I controlli che si possono esprimere come regola non vengono delegati al modello, perché un modello linguistico valuta solo quello che ha davanti, e in questo caso non aveva abbastanza informazioni per accorgersi dell'errore.

8 Risultati

8.1 Il confronto con il sistema di partenza

Il programma mette a disposizione un'opzione che risponde alla stessa domanda con i due sistemi, uno dopo l'altro. Entrambi usano lo stesso modello linguistico, quindi l'unica differenza fra le due risposte è l'architettura. L'esempio seguente usa il database fornito con il progetto, che contiene tre clienti di cui uno italiano.

```
Question: Quanti clienti italiani ci sono?

--- before: fixed pipeline, retries only on SQL errors ---
  SELECT COUNT(*) AS num_italian_customers
FROM customers
WHERE country = 'Italy'
      num_italian_customers
0
(1 rows)

--- after: agent that inspects the database ---
C'è 1 cliente italiano (country = 'IT').

steps taken:
  list_tables({})
  describe_table({'table': 'customers'})
  sample_rows({'table': 'customers'})
  run_select({'sql': "SELECT COUNT(*) AS n FROM customers
                    WHERE country = 'IT'"})
```

Listing 5: Le due risposte alla stessa domanda, con la traccia degli strumenti usati dall'agente. Le etichette stampate dal programma sono in inglese.

Il sistema di partenza scrive un filtro sul nome esteso del paese, ottiene zero righe e si ferma, perché l'esecuzione non ha prodotto errori. Il sistema agencico guarda prima quali tabelle esistono, poi le colonne di `customers`, poi alcune righe reali, e solo a quel punto scrive la query usando il valore che ha trovato nei dati.

La traccia mostra che nessuno di quei passi è previsto dal codice. È il modello a scegliere quali strumenti usare e in quale ordine.

8.2 Le prove sul database esteso

Il database fornito con il progetto contiene due tabelle e sette righe in tutto, il che basta per la dimostrazione ma non per verificare che il sistema regga su dati realistici. È stato quindi costruito un secondo database, con quattro tabelle e circa millecinquecento righe, generato da un seme fisso e quindi riproducibile in modo identico. Al suo interno sono state inserite apposta alcune difficoltà che si incontrano nei dati veri.

Difficoltà	Perché conta
country contiene codici	un filtro sul nome esteso restituisce zero righe senza errori
status contiene lettere	il significato dei codici va ricavato guardando i dati
colonne che possono essere nulle	contare una colonna e contare le righe dà risultati diversi
name esiste in due tabelle	le unioni fra tabelle devono indicare quale colonna usare
unit_price esiste in due tabelle	il fatturato va calcolato sul prezzo dell'ordine, non su quello corrente
circa millecinquecento righe	i risultati vengono troncati e non vanno contati come totali

Table 6: Le difficoltà inserite nel database di valutazione.

Sono state poi provate cinque domande di difficoltà crescente, ripetute tre volte ciascuna su entrambi i sistemi per un totale di trenta esecuzioni. La risposta è stata confrontata con il valore calcolato direttamente sul database. Il confronto è automatico e tiene conto del fatto che la risposta è in linguaggio naturale, quindi estrae i numeri dal testo invece di cercare una corrispondenza esatta. Anche la valutazione è uno script del progetto, e può essere rieseguita.

Domanda	Atteso	Agentico	Di partenza
Quanti clienti italiani ci sono	40	3/3	0/3
Quanti ordini sono stati annullati	36	3/3	0/3
Quanti clienti non hanno email	20	3/3	3/3
Quanti clienti tedeschi ci sono	24	3/3	0/3
Fatturato esclusi gli annullati	741458,03	3/3	0/3
Totale		15/15	3/15

Table 7: Risposte corrette su tre ripetizioni per domanda.

L'unica domanda a cui il sistema di partenza risponde correttamente è quella sui clienti senza email, che è anche l'unica a non richiedere di sapere come sono scritti i valori nelle colonne. Nelle altre il filtro viene costruito sul nome esteso del paese o sullo stato dell'ordine, e il risultato è zero.

Il caso del fatturato è diverso e merita attenzione. Il sistema di partenza non restituisce zero ma 807486,86, cioè il fatturato comprensivo degli ordini annullati, perché non ha modo di sapere che la lettera X indica un annullamento. La risposta è plausibile, non è segnalata da alcun errore ed è sbagliata di circa sessantaseimila. È il caso peggiore dei due, perché nulla invita a controllarla.

La domanda sul fatturato è anche la più impegnativa per l'agente, perché richiede di unire tre tabelle, di escludere gli ordini annullati riconoscendo che lo stato è indicato da una lettera, e di usare il prezzo registrato sulla riga dell'ordine invece di quello corrente del prodotto. Il significato dei codici di stato non è scritto da nessuna parte, e l'agente lo ha ricavato campionando la tabella.

Le domande sono state però scritte da chi ha costruito il database di valutazione, quindi il confronto misura quanto i due sistemi affrontano le difficoltà previste e non quanto se la caverebbero su dati raccolti da altri. Il limite è ripreso nel capitolo 9.

8.3 I test automatici

Il progetto ha centoventisette test automatici, che si eseguono senza bisogno di una chiave API. Coprono il controllo sulle scritture, comprese le query che aggiravano quello del sistema di partenza, la lettura dello schema, il comportamento degli strumenti e l'interfaccia a riga di comando.

La parte più utile riguarda il grafo, provato sostituendo al modello linguistico un oggetto che restituisce risposte prestabilite. In questo modo l'approvazione, il rifiuto con ritorno all'agente, il limite alle revisioni, la separazione fra conversazioni diverse e l'autocorrezione su un errore SQL sono verificati in modo deterministico, senza dipendere da cosa risponde il modello.

9 Limiti

Il sistema funziona solo su SQLite. La lettura dello schema usa le istruzioni **PRAGMA**, che sono specifiche di quel motore, e l'apertura in sola lettura sfrutta un parametro della connessione che altri database non hanno. Portarlo su PostgreSQL o MySQL richiederebbe di riscrivere la lettura dello schema e di trovare un modo equivalente per impedire le scritture, mentre il resto dell'architettura resterebbe invariato.

Il database usato per le prove del capitolo 8 è stato costruito da chi ha poi valutato il sistema, e le difficoltà che contiene sono le stesse che il sistema sa affrontare. I risultati mostrano quindi che l'agente gestisce correttamente i casi previsti, non che li gestirebbe su dati raccolti da altri. È il limite metodologico più serio del lavoro.

Le prove del capitolo 8 sono cinque domande ripetute tre volte, quindi quindici esecuzioni per sistema. È un campione piccolo. Il risultato dice che su quelle domande il sistema agentico è stato stabile, ma con tre ripetizioni un comportamento che fallisce di rado potrebbe non essersi mai presentato. Per una misura più solida servirebbero più domande e più ripetizioni.

Non è infine gestito il caso di uno schema troppo grande per entrare nel contesto del modello. Sui database usati la descrizione di tutte le tabelle occupa poco spazio, ma con centinaia di tabelle l'agente non avrebbe modo di capire quali siano rilevanti per la domanda, e servirebbe un passo che le seleziona prima.

10 Conclusioni e sviluppi futuri

Il lavoro è partito da un sistema che traduceva una domanda in SQL e ritentava quando il database restituiva un errore. Il risultato è un sistema in cui il modello decide da solo quali strumenti usare e in quale ordine, guarda lo schema e i dati prima di scrivere la query, e sottopone la propria risposta a un controllo prima di restituirla. Il ciclo di orchestrazione scritto in Python non esiste più, e la sequenza delle operazioni non è più prevedibile leggendo il codice.

Il meccanismo di autocorrezione del sistema di partenza non è stato però buttato. È stato riportato dentro il grafo, dove è diventato uno dei due cicli.

Dal capitolo 7 è uscito il risultato più utile del lavoro, che è anche un risultato negativo. Affidare a un modello linguistico la verifica della correttezza di una risposta non ha funzionato proprio nel caso che contava, perché il verificatore non disponeva delle informazioni necessarie per accorgersi dell'errore. Il controllo è stato riscritto come regola dentro lo strumento, e al modello

è rimasta la responsabilità dei soli casi che richiedono interpretazione. Il criterio che ne deriva è di non delegare al modello ciò che si può decidere in modo meccanico.

Resta utile distinguere due tipi di garanzia. I centoventisette test provano in modo deterministico il comportamento del grafo e dei controlli, perché sostituiscono al modello un oggetto che risponde in modo prestabilito. Le prove del capitolo 8 invece osservano come si comporta il sistema con un modello vero, su un numero limitato di esecuzioni. Le prime sono garanzie, le seconde sono osservazioni, e confonderle porterebbe ad attribuire al sistema un'affidabilità che non è stata misurata.

Gli sviluppi più sensati partono dai limiti del capitolo 9. Una valutazione su benchmark pubblici toglierebbe il problema di avere scritto le domande su misura per il sistema. Il supporto ad altri motori di database richiede di riscrivere la lettura dello schema e il controllo sulle scritture, lasciando il grafo invariato. Un'architettura con più agenti coordinati da un supervisore, infine, si innesterebbe sul grafo esistente aggiungendo nodi, senza doverlo ripensare.